

Лекция 6. Обеспечение безопасности ОС Unix (Linux)

Учебные вопросы:

1. Механизмы защиты ОС семейства Unix
2. Аудит в ОС семейства UNIX

1. Механизмы защиты ОС семейства Unix

Защита ОС семейства Unix в общем случае базируется на трех основных механизмах:

1. идентификации и аутентификация пользователя при входе в систему;
2. разграничении прав доступа к файловой системе, в основе которого

лежит реализация дискреционной модели доступа;

3. аудит (регистрация событий).

При этом отметим, что для различных клонов ОС семейства Unix возможности механизмов защиты могут незначительно различаться.

Аутентификация в UNIX

Список пользователей UNIX хранится в текстовом файле `/etc/passwd`, формат которого допускает его редактирование обычным текстовым редактором. В старых версиях UNIX зашифрованные образы паролей хранились в том же файле. В большинстве современных UNIX-систем образы паролей хранятся в отдельном файле, также расположенном в каталоге `/etc`. Имя этого файла различается в разных версиях UNIX, чаще всего применяется имя `/etc/shadow`. В некоторых UNIX-системах пароль каждого



пользователя хранится в своем отдельном файле, например, в ALT Linux пароль пользователя username хранится в файле /etc/tcb/username/shadow.

Каждая строка файла /etc/passwd соответствует одному пользователю или псевдопользователю. Она включает в себя следующие семь полей, разделенных двоеточиями:

- ~ имя пользователя (login);
- ~ хэш-образ пароля пользователя или признак того, что хэш-образ пароля хранится в другом файле, общем для всех пользователей (*) или признак того, что хэш-образ пароля хранится в отдельном файле (x);
- ~ UID пользователя;
- ~ GID первичной группы пользователя;
- ~ комментарий к учетной записи пользователя (произвольная текстовая строка);
- ~ домашняя директория пользователя;
- ~ командный интерпретатор, который должен запускаться на терминалах, на которых работает данный пользователь (для псевдопользователей обычно указывается /dev/null). Например, root:x:0:0:System Administrator:/root:/bin/bash.

Управление списком пользователей, как правило, осуществляется не путем ручного редактирования соответствующих файлов, а с использованием специальных утилит командной строки user add, userdel, usermod, groupadd, groupdel, groupmod, либо графических утилит, оснасток панели управления и т. п.

Некоторые загрузчики Linux позволяют установить пароль, запрашиваемый при загрузке системы. Так, при работе с LILO (Linux Loader) можно использовать параметры: «установить пароль для

начальной загрузки» и «разрешение загрузки» после указания определенных опций в ответ на запрос LILO.

Хранение отпечатка пароля в файле `/etc/passwd` применялось в старых версиях UNIX систем, при этом оставалась возможность оффлайнного подбора пароля. Учитывая, что содержимое файла `/etc/passwd` имеет текстовый формат, а сам файл – общедоступен, такой способ хранения аутентификационной информации нельзя считать надежным. Поэтому для Linux были разработаны дополнительные механизмы защиты.

Механизм теневых паролей. Файл `/etc/passwd` больше не содержит отпечатка пароля, вместо него помещается значение, указывающее, что отпечаток пароля хранится в специальном парольном файле. Права на чтение парольного файла `/etc/shadow` принадлежат только суперпользователю.

Хранимые в файле отпечатки пользовательских паролей получены с помощью функции `crypt`, допускающей как шифрование паролей с помощью любых методов, поддерживаемых криптографической библиотекой. Например, в Linux Debian/Ubuntu библиотека `libcrypt` поддерживает хэш, основанный на современном шифре Blowfish, или бесключевые хэш-функции с подмешиванием случайной информации – соли, при этом используется многократное (порядка 1000 раз) применение алгоритма, что замедляет вычисление отпечатка, а значит и снижает вероятность успешного подбора паролей. Для реализации механизма теневых паролей в Linux используется набор программных средств Shadow Suite. В большинстве современных дистрибутивов Linux механизм теневых паролей задействован по умолчанию. Если же теневые пароли не используется, для хэширования паролей следует применять метод с использованием современных шифров.

Архитектура *Pluggable Authentication Module (PAM)* разработана Open Software Foundation (OSF) в качестве подсистемы аутентификации для современных UNIX/Linux-систем. Программные модули, входящие в подсистему аутентификации, делятся на две основные группы:

- ~ клиенты (приложения и демоны), пользующиеся услугами PAM (login, passwd, rlogin, telnetd, ftpd и т.п.);
- ~ программные модули, предоставляющие услуги PAM. Обычно эти модули представляют собой библиотеки, размещаемые в директории `/lib/security`.

С точки зрения клиента аутентификации его взаимодействие с PAM реализуется посредством так называемых примитивов – системных вызовов, передающих управление соответствующим модулям PAM. Поддерживаются шесть основных примитивов, сгруппированных в четыре подсистемы:

- ~ `pam_authenticate` (подсистема `auth`) – аутентифицировать пользователя;
- ~ `pam_jsetcred` (подсистема `auth`) – авторизовать пользователя (установить UID, идентификаторы групп, квоты ресурсов и т.д.);
- ~ `pam_acct_mgmt` (подсистема `account`) – проверить, доступна ли учетная запись пользователя для авторизации (не устарел ли пароль, не заблокирована ли учетная запись и т.п.);
- ~ `pam_openjsession` (подсистема `session`) – начать сеанс работы пользователя с операционной системой;
- ~ `pam_closejsession` (подсистема `session`) – завершить сеанс работы пользователя с операционной системой;
- ~ `pam_chauthtok` (подсистема `password`) – назначить пользователю новые аутентификационные данные.

Вся техническая реализация процедуры аутентификации пользователя инкапсулирована внутри PAM, клиенту выдается лишь самая общая информация о результатах выполнения PAM того или иного примитива.

Каждый модуль PAM должен содержать функции-обработчики примитивов хотя бы одной подсистемы. Когда клиентская программа вызывает тот или иной примитив, PAM обращается к функциям-обработчикам данного примитива одного или нескольких своих модулей и принимает решение на основании возвращаемых ими результатов.

Конкретный порядок того, какие функции каких модулей должны вызываться и как должны интерпретироваться результаты их вызова, определяется содержимым конфигурационных файлов PAM.

В ранних версиях PAM вся конфигурация PAM описывалась единственным файлом `/etc/pam.conf`. В более поздних версиях каждой клиентской программе сопоставляется индивидуальный файл конфигурации, расположенный в директории `/etc/pam.d`. Имя этого файла обычно совпадает с

именем клиента, так, например, файл конфигурации службы SSH обычно имеет имя `/etc/pam.d/sshd`. В каждом файле конфигурации содержится последовательность строк, определяющих, какие PAM-модули должны использоваться клиентом и каким образом это должно осуществляться. Конфигурация PAM, используемая клиентскими программами по умолчанию, описывается в файле `/etc/pam.d/other`.

Если, например, в некоторой конфигурации ОС необходимо полностью запретить выполнение команды `su`, достаточно вписать в начало файла `/etc/pam.d/su` строку: `auth requisite pam_deny.so`.

Модули PAM позволяют реализовать в ОС также аутентификацию

использованием внешних электронных носителей информации (например, `ram_usb.so`), а также биометрическую аутентификацию (например, `ramJxprint.so`). С помощью модуля `ramJistfile.so` можно реализовать в одной ОС различные схемы аутентификации для конкретных списков пользователей, групп пользователей, хостов и т.п., например, следующая строка конфигурационного файла:

```
~      auth    sufficient    pamJistfile.so    item=user    sense=deny  
file=/etc/users deny
```

запрещает вход в систему пользователям, имена которых перечислены в файле `/etc/user.deny`.

Таким образом, архитектура PAM позволяет администратору UNIX системы гибко настраивать подсистему аутентификации в соответствии с реальными потребностями конкретной вычислительной сети и конкретного экземпляра ОС. Все, что нужно сделать администратору, – установить в системе необходимые модули PAM и обеспечить корректное взаимодействие между ними. В результате подсистема аутентификации может быть легко адаптирована к самым разным технологиям аутентификации, принятым в конкретной организации.

Сетевая защита

Сетевой защите в клонах ОС UNIX последних версий уделяется значительное внимание.

Так, в ОС Linux для обеспечения сетевой безопасности применяется несколько эффективных средств:

```
~      защищенная оболочка ssh для предотвращения атак, в  
которых для получения паролей используются анализаторы  
протоколов;
```

```
~      программы tcp_wrappers для ограничения доступа к  
различным службам компьютера;
```

- ~ межсетевой экран netfilter для ядер Linux, начиная с версии 2.4;
- ~ сетевые сканеры для выявления уязвимых мест компьютера;
- ~ демон tcprd для обнаружения попыток сканирования портов со стороны злоумышленников (в дополнение к этому средству полезно регулярно просматривать файлы системного журнала);
- ~ система шифрования PGP (Pretty Good Privacy);
- ~ программа stelnets (защищенная версия хорошо известной программы telnet);
- ~ программа qmail (защищенная доставка электронной почты);
- ~ программа ipfwadm для настройки межсетевых экранов (firewall);
- ~ режим проверки паролей входных соединений для систем, разрешающих подключение по внешним коммутируемым линиям связи или локальной сети.

Управление доступом к данным

В ОС семейства UNIX к объектам доступа относятся файлы, директории, ссылки (links), устройства и именованные каналы (named pipes). В большинстве современных версий UNIX выполняющиеся процессы также рассматриваются как объекты доступа, размещенные в специальной директории /proc. Атрибуты защиты этих объектов могут учитываться при доступе отладчика к адресному пространству процесса, при отправке процессу сигналов и т. п.

К субъектам доступа в UNIX относятся пользователи (включая псевдопользователей) и группы пользователей.

Среди пользователей UNIX выделяется предопределенный пользователь **root**, обладающий в ОС абсолютными полномочиями и

имеющий возможность игнорировать любые принятые в системе правила управления доступом.

Пользователи и группы пользователей идентифицируются в UNIX числовыми идентификаторами, уникальными в пределах одного экземпляра ОС. Пользователь `root` всегда имеет идентификатор 0. Идентификаторы пользователей обозначаются стандартным обозначением UID (User ID), идентификаторы групп – GID (Group ID).

Все методы доступа ко всем объектам регулируются в UNIX тремя правами доступа:

- ~ Read (R) – чтение;
- ~ Write (W) – запись;
- ~ Execute (X) – выполнение.

Для файлов смысл всех трех прав доступа вполне очевиден. Для каталогов право Read дает возможность получения списка вложенных файлов и подкаталогов, право Write – возможность создания, переименования и/или удаления вложенных файлов и подкаталогов, право Execute – переход в каталог (команда `cd`). Для устройств каждое право доступа дает возможность обращения к соответствующей функции драйвера, обслуживающего данное устройство. Для именованного канала право Read требуется для получения информации, право Write – для отправки, право Execute не определено.

Понятие привилегии субъекта доступа в ранних версиях UNIX систем

отсутствовало. В наиболее распространенной на сегодняшний день ее реализации, применяемой, в частности, в Linux, привилегии называются возможностями (capabilities). Так, в ядре Linux 2.6.34 определены, например,

следующие возможности субъектов доступа:

- ~ CAP.CHOWN – позволяет модифицировать идентификаторы владельца или группы владельца любого объекта;

~ CAP.DAC.OVERRIDE – позволяет игнорировать любые ограничения на доступ к объектам, задаваемые списками управления доступом;

~ CAP.SETPCAP – позволяет включать или выключать любую привилегию у любого процесса;

~ CAP_NET_BROADCAST – позволяет отправлять широковещательные сетевые пакеты;

~ CAP_SYS_MODULE – позволяет загружать и выгружать модули расширения ядра ОС;

~ CAP_SYS_RAWIO – позволяет выполнять низкоуровневые операции ввода-вывода (ioperm, iopl), взаимодействовать с USB-устройствами посредством виртуального файла /proc/bus/usb;

~ CAP_SYS_BOOT – позволяет перезагружать ОС;

~ CAP_SYS_TIME – позволяет управлять системными таймерами;

~ CAP_SYS_TTY_CONFIG – позволяет управлять консольными терминалами.

Каждому процессу UNIX присваиваются четыре числовых идентификатора:

~ UID – идентификатор пользователя, породившего данный процесс;

~ GID – идентификатор группы пользователя, породившего данный процесс;

~ EUID – эффективный идентификатор пользователя;

~ EGID – эффективный идентификатор группы пользователя.

Обычно EUID совпадает с UID, а EGID – с GID.

Каждый объект UNIX может иметь атрибуты защиты, включающие

в

себя следующие три элемента:

- ~ идентификатор (UID) владельца объекта;
- ~ идентификатор группы (GID) владельца объекта;
- ~ вектор доступа.

Вектор доступа включает в себя следующие элементы:

- ~ является ли объект каталогом – 1 бит (D);
- ~ бит SUID;
- ~ бит SGID;
- ~ бит sticky;
- ~ права доступа владельца – 3 бита (RWX);
- ~ права доступа пользователей, входящих в группу владельца – 3 бита (RWX);
- ~ права доступа всех остальных пользователей – 3 бита (RWX).

Проверка прав доступа субъекта к объекту осуществляется по следующему алгоритму.

1. Если EUID процесса равен нулю (т.е. процесс выполняется от имени суперпользователя), доступ к объекту предоставляется без каких бы

то ни было дополнительных проверок.

2. Если EUID процесса совпадает с UID владельца объекта, права доступа, запрошенные процессом, сравниваются с правами, разрешенными владельцу данного объекта. Если все права, запрошенные процессом, разрешены владельцу объекта, доступ разрешен, в противном случае – запрещен.

3. Если EUID процесса отличается от UID владельца объекта, но EGID процесса совпадает с GID владельца объекта, права доступа, запрошенные процессом, сравниваются с правами, разрешенными пользователям, входящим в группу владельца объекта. Если все права, запрошенные процессом, разрешены группе владельца объекта, доступ разрешен, в противном случае – запрещен.

4. Если EUID процесса отличается от UID владельца объекта и EGID процесса отличается от GID владельца объекта, права доступа, запрошенные процессом, сравниваются с правами, разрешенными пользователям, не входящим в группу владельца объекта (всем остальным пользователям). Если все права, запрошенные процессом, разрешены пользователям, не входящим в группу владельца объекта, доступ разрешен, в противном случае – запрещен.

Бит sticky в современных UNIX-системах применяется только в отношении директорий. Если в векторе доступа директории установлен данный бит, удаление файлов и поддиректорий разрешается только их владельцам.

Наличие в UNIX суперпользователя root с ничем не ограниченными полномочиями создает серьезные проблемы не только для безопасности, но и для надежности ОС. Любая ошибка, допущенная суперпользователем при работе с командной строкой, может стать фатальной. Если, например, в команде `rm -rf /*.bak` ошибочно поставить пробел между звездочкой и точкой, сама ОС и все хранящиеся в ней данные будут немедленно и необратимо уничтожены. Большинство администраторов UNIX в настоящее время избегают авторизоваться в ОС в качестве суперпользователя на протяжении всего сеанса работы с системой. Если администратору необходимо выполнить какие-то действия, требующие повышенных полномочий, он может воспользоваться утилитой командной строки `su`. Эта утилита получает имя и пароль пользователя (на практике чаще всего используется имя `root`, его можно не вводить, считается, что оно задано по умолчанию), и, если введены корректные данные, инициирует сеанс работы указанного пользователя на текущем терминале, запуская новую копию командного интерпретатора.

Прекращение сеанса работы пользователя, инициированного командой `su`, происходит естественным образом при завершении командного интерпретатора, например, командой `exit`.

Во многих UNIX-системах пользоваться утилитой `su` по умолчанию (если администратор не менял настройки аутентификации для программы `su`) разрешено только пользователям, входящим в группу `wheel`. Иногда эту группу неформально сравнивают с группой администраторов Windows, а использование утилиты `su` – с запуском программы Windows на повышенном уровне мандатной целостности.

На практике вместо `su` чаще применяется утилита `sudo`, реализующая выполнение от имени суперпользователя (или другого пользователя, если указан флаг `-u`) команды, являющейся параметром команды `sudo`. Например, команда `sudo mc` запускает от имени суперпользователя файловый менеджер Midnight Commander.

При первом вводе `sudo` пользователь должен повторно ввести свой пароль. После этого в течение некоторого времени (по умолчанию пять минут) пользователь может использовать команду `sudo` без пароля. Порядок использования команды `sudo` описывается конфигурационным файлом `/etc/sudoers`. Пример: суперпользователь `root` может выполнять через `sudo` любые команды, не вводя никаких паролей:

```
root ALL = NOPASSWD: ALL
```

2. Аудит в ОС семейства UNIX

В современных ОС семейства UNIX реализуются два принципиально различных подхода к организации аудита. Первый из них основан на применении традиционных для UNIX демонов регистрации событий `syslogd` и

klogd. Второй основан на переносе в среду UNIX архитектуры и интерфейсов подсистемы аудита, изначально свойственных ОС семейства Windows.



В состав большинства современных UNIX-систем могут включаться обе подсистемы аудита, при этом они могут работать как независимо одна от другой, так и во взаимодействии.

Демон syslogd предоставляет услуги прикладным и системным программам, демон klogd реализует аудит операций, выполняемых ядром ОС. Основным назначением этих демонов является поддержка единой в масштабах ОС политики аудита, предписывающей выполнение определенных действий для каждого типа регистрируемых событий. Каждый раз, когда прикладная или системная программы выполняет потенциально аудируемое действие, информация об этом действии передается демону syslogd, тот сверяется с текущей политикой аудита и принимает решение о порядке регистрации данного события. Получение информации демоном syslogd обычно реализуется через сокет /dev/log (для локальных клиентов) или UDP-порт 514 (для удаленных клиентов). Демон klogd получает информацию через специальный файл /proc/kmsg или системный вызов ядра sys_syslog. Идентификаторы процессов демонов аудита хранятся в десятичном виде в текстовых файлах /var/run/syslogd.pid и /var/run/klogd.pid соответственно.

Действующая политика аудита описывается текстовым файлом /etc/syslog.conf.

В большинстве UNIX-систем большая часть сообщений аудита традиционно записывается в файл `/var/log/messages` или `/var/adm/log/messages`.

Для хранения сообщений о попытках аутентификации традиционно используется файл `/var/log/auth.log` или `/var/adm/loginlog`.

При обновлении файла `syslog.conf` новая политика аудита вступает в силу только после перезагрузки демона `syslogd` или после того, как этот демон получит сигнал `SIGHUP`. Прикладные программы UNIX интерпретируют данный сигнал как обрыв связи удаленного клиента с терминалом и обычно, получив его, аварийно завершают работу, но демон `syslogd` воспринимает сигнал `SIGHUP` иначе – как требование обновить политику аудита.

Журналы аудита UNIX представляют собой обычные текстовые файлы. Их просмотр и анализ может проводиться как обычными утилитами просмотра текстов, так и более продвинутыми программными средствами, облегчающими работу с аудитом.

Удаление из файлов аудита устаревшей информации традиционно реализуется путем регулярного запуска демоном `sgon` специальной утилиты `logrotate`, конфигурация которой описывается файлом `/etc/logrotate.conf`.

Серьезным недостатком подсистемы аудита UNIX является то, что доступ к демону `syslogd` в общем случае предоставляется любым программам, как системным, так и прикладным. При этом клиентская программа не только передает демону текст сообщения, но и самостоятельно указывает источник и приоритет регистрируемого события. Более того, существует специальная утилита командной строки `logger`, позволяющая обычному непривилегированному

пользователю передать демону syslogd произвольную информацию от имени произвольной подсистемы ОС и присвоить этой информации произвольный уровень значимости, например, имитировать нарушение правил безопасности другим пользователем.

Auditd. Механизм регистрации событий, реализуемый демонами syslogd и klogd, в основном предназначен не столько для поддержания безопасности системы, сколько для выявления программных и аппаратных неисправностей, тестирования и отладки новых компонент ОС и т.п. Применять данный механизм для регистрации событий, связанных с безопасностью системы, не очень удобно, и в некоторых ОС, принадлежащих к семейству UNIX, для этой задачи предназначен отдельный, дополнительный демон аудита, которому чаще всего назначается имя auditd. Концептуально реализации данного демона в разных UNIX-системах обычно очень похожи на реализацию подсистемы аудита в Windows. Однако технические детали разных реализаций могут очень сильно отличаться одна от другой.

Подсистема контроля целостности

Развитый механизм контроля целостности системных файлов ОС Linux – IMA (Integrity Measurement Architecture), доступен для ядра версии 2.6.30 и выше. Технология IMA включена не во все дистрибутивы, поэтому для ее использования может потребоваться пересборка ядра.

Подсистема IMA отвечает за вычисление хэш-значений файлов и программ при их открытии до того, как их содержимое будет доступно для чтения или выполнения, а также поддерживает проверку их соответствия заранее заданному списку и создание отчетов.

Когда система впервые устанавливается в известном доверенном состоянии, записывается список «хороших» значений хеш-функции. Во время обычной работы системы IMA поддерживает список текущих файловых хэшей. Затем этот список можно сравнить с первоначально

записанным списком «хороших» хэшей, чтобы обнаружить любые необоснованные изменения. Технология IMA изначально использовалась только для дальнейшей аттестации системы, то есть была пассивным средством защиты информации.

В Linux системах возможна проверка целостности установленных в системе пакетов. Это позволяет убедиться в том, что файлы, установленные из пакета, не были изменены после инсталляции. В Red Hat Linux и других дистрибутивах, использующих для установки пакетов RPM (RPM Package Manager) для этих целей используется опция `--verify (-V)` одноименной команды: `rpm --verify --all` (или `rpm -Va`). Можно выполнить также проверку файлов отдельного пакета или пакета, которому принадлежит указанный файл. При проверке пакета информация об установленных из пакета файлах сравнивается с информацией из оригинального пакета. Помимо прочего, при такой проверке проверяется размер, сумма MD5, разрешения, тип, владелец и группа владельца каждого файла. Результатом выполнения команды будет список кодов найденных ошибок с указанием полного пути файла.

Функции криптографической защиты

В ОС Linux поддержка шифрования на уровне файловых систем введена сравнительно недавно – начиная с версии ядра 4.1 и реализуется `fscrypt` – библиотекой, в которую могут подключаться файловые системы для поддержки прозрачного шифрования каталогов.

Как правило, возможности шифрования используются для защиты домашней папки пользователя. Встроенное шифрование `ext4` осуществляется с помощью утилиты `e4crypt`, входящей в пакет `e2fsprogs`. `fscrypt` шифрует содержимое файлов, но оставляет открытыми метаданные файловой системы за исключением имен

файлов (например, размеры файлов, права доступа, временные метки и расширенные атрибуты).

Реализация шифрования на уровне файловых систем в ОС Linux имеет ряд особенностей. Так, не поддерживается шифрование уже существующих файлов. Вместо этого доступна маркировка пустого каталога как зашифрованного. Затем, после того как пользовательское пространство предоставит ключ, все обычные файлы, каталоги и символические ссылки, созданные в этом каталоге и его подкаталогах, будут прозрачно зашифрованы. Имена файлов в зашифрованном каталоге шифруются, как и содержимое файлов. Остальная метаинформация файловой системы, включая наличие и расположение нераспределенных блоков, остается открытой. Если файл из зашифрованной папки был открыт пользователем, его содержимое и имя кэшируется и становится доступным другим пользователям в той же системе, независимо от видимости ключа. Поэтому защита открытых файлов может быть обеспечена только разграничением прав доступа. Кроме того, если к зашифрованному файлу недавно обращались, ключи остаются в памяти (а, следовательно, имена и открытое содержимое файлов можно восстановить) по крайней мере, до тех пор, пока файловая система не будет размонтирована или не будут удалены кэши файловой системы. Также в настоящее время не реализована защита от расшифрования файлов другого пользователя с неправильно указанным паролем (что может сделать их временно недоступными для законного владельца).

Для получения доступа к файлам из зашифрованного каталога в начале каждого сеанса работы ОС Linux пользователь должен сгенерировать ключи.

При использовании стандартных параметров генерации ключи, получаемые разными пользователями на основе одного и того же пароля, совпадут.